

Gruppe *Nummer*

Pflichtenheft für das Thema „*Klick-Tool*“

Betreuer: *Prof. Dr. Andreas Nüchter*

Tom Lerzer, Fabia Frank, Luis Ángel Álvarez Gil

SS2026

1 Zielbestimmung

1.1 Muss-Kriterien

- Laden von Bilddaten in jpg und png Format
- Internes Erstellen von png-Dateien bei Eingabe von 3DTK-spezifischen 3D-Dateiformaten
- Implementieren eines GUI zur erleichterten Ausführung der implementierten Funktionalitäten
- Platzieren von maximal zwei der geladenen Bilder nebeneinander
- Zoom-Funktion für X-fachen Zoom jedes geladenen Bildes
- Anwählen einzelner Koordinaten in einem Bild mit Subpixelgenauigkeit von mindestens 0,5 Pixel
- Anwählen korrespondierender Koordinaten in zwei Bildern mit Subpixelgenauigkeit von mindestens 0,5 Pixel
- Visualisierung gewählter Koordinaten
- Visualisierung der Korrespondenz gewählter Koordinaten
- Ausgabe gewählter 2D-Koordinaten in Form $(x_1, y_1), \dots, (x_n, y_n)$ für einfache bzw. $(x_{1.1}, y_{1.1}, y_{1.2}, y_{1.2}), \dots, (x_{n.1}, y_{n.1}, x_{n.2}, y_{n.2})$ in .txt-Dateiformat
- Integration der implementierten Funktionalitäten in 3DTK-Toolkit
- Implementieren von Kommandozeilenparametern sowie deren Beschreibungen für das Aufrufen der implementierten Funktionalitäten
- Erstellen eines Entwicklerhandbuchs für die implementierten Funktionalitäten
- Erstellen eines Benutzerhandbuchs für die implementierten Funktionalitäten

1.1.1 Halbzeit-Ziele

- Laden von Bilddaten in jpg und png Format
- Internes Erstellen von png-Dateien bei Eingabe von 3DTK-spezifischen 3D-Dateiformaten
- Implementieren eines GUI zur erleichterten Ausführung der implementierten Funktionalitäten
- Platzieren von maximal zwei der geladenen Bilder nebeneinander
- Zoom-Funktion für X-fachen Zoom jedes geladenen Bildes
- Anwählen einzelner Koordinaten in einem Bild mit Subpixelgenauigkeit von mindestens 0,5 Pixel
- Anwählen korrespondierender Koordinaten in zwei Bildern mit Subpixelgenauigkeit von mindestens 0,5 Pixel

1.1.2 Zuständigkeiten

- Erstellen der grafischen Nutzeroberfläche für Platzierung der Bilder sowie Visualisierung gewählter Koordinaten:
- Funktionalität zum Laden der Bilddateien und Umwandeln von 3DTK-spezifischen 3D-Formaten:
- Implementieren der Zoom-Funktion:
- Funktionalität zum Anwählen und Ausgeben von Koordinaten bzw. korrespondierenden Koordinaten:
- Integration der implementierten Funktionalitäten in 3DTK:
- Implementieren der Kommandozeilenparameter incl. Beschreibung:
- Erstellen Entwicklerhandbuch:
- Erstellen Benutzerhandbuch:

1.2 Kann-Kriterien

- Automatische Wahl der optimalen Anordnung der Bilder (nebeneinander oder übereinander) aufgrund Bildgröße und Bildschirmauflösung
- Ausblenden einzelner Korrespondenzlinien zwischen Punkten
- Anzeige Koordinaten der gewählten Punkte
- Undo-Funktion für ausgewählte Punkte
- Frei positionierbare Lupenansicht

1.3 Abgrenzungskriterien

- Anwahl korrespondierender Punkte wird ausschließlich manuell durchgeführt

2 Einsatz

2.1 Anwendungsbereiche

- Bildverarbeitung
- Robotik
- Photogrammetrie

2.2 Zielgruppe

Das Projekt soll durch Studierende und Lehrende der Universität Würzburg im Rahmen der Robotik-Forschung, der Bildverarbeitung sowie allgemein im Bereich Informatik zum Einsatz gebracht werden. Für weitere interessierte Personen ist das Projekt auf GitHub zugänglich.

3 Umgebung

3.1 Software

- Programmiersprache: C++
- Bildverarbeitung: OpenCV **Version XXX**, **Link**
- Nutzeroberfläche: Dear ImGui **Version XXX**, **Link**
- 3D-Framework: 3DTK
- Entwicklungsumgebung: Visual Studio Code
- Compiler: g++
- Build-System: CMake, mindestens Version 3.4
- Versionsverwaltung: git, Version 2.51
- Fensterverwaltung: GLFW, Version 3.4
- Grafik-API: OpenGL

3.2 Hardware

Die Hardware muss alle Anforderungen der im vorherigen Punkt genannten Software erfüllen.

4 Funktionalität

- Nutzer starten das Programm via Terminal. Beim Start des Programms werden ein oder zwei Parameter in den unter *Daten* angegebenen Formaten eingegeben
- Auf dem angezeigten Bild bzw. den angezeigten Bildern kann vor der weiteren Bearbeitung bis zu **X-Mal** hineingezoomt werden
- Nutzer wählen beliebig viele einzelne oder korrespondierende Punkte aus
- Programm zeigt die angewählten Punkte und deren Korrespondenzen im GUI an
- Durch Anwählen des Buttons **Ausgabe** initiieren Nutzer die Ausgabe der gewählten Punkte in .txt-Format

5 Daten

- **Eingabe:** Dateien im png oder jpg-Format sowie 3DTK-spezifische Formate, die durch Anwendung der *scan_to_panorama*-Funktion in anzeigbare png-Dateien umgewandelt werden
- **Ausgabe:** .txt-Datei mit Koordinaten in Form $(x_1, y_1), \dots, (x_n, y_n)$ für einfache bzw. $(x_{1.1}, y_{1.1}, y_{1.2}, y_{1.2}), \dots, (x_{n.1}, y_{n.1}, x_{n.2}, y_{n.2})$ für korrespondierende Punkte
- **3DTK-spezifische Formate:** rexp, uos, uos_map, uos_rgb, uos_frames, uos_map_frames, old, rts, rts_map, ifp, riegl_txt, riegl_rgb, riegl_bin, zahn, ply, las

6 Leistungen

Das Laden der Bilddateien, die Umwandlung der 3DTK-spezifischen Datenformate, die Zoom-Funktionalität sowie Auswahl und Ausgabe der gewählten Koordinaten soll weitestgehend performant sein und so in einer angemessenen Zeit geschehen.

7 Benutzungsoberfläche

Dieser Abschnitt kann bei Themen ohne GUI weggelassen werden; ansonsten grobe Skizzen der wichtigsten GUI-Ansichten mit jeweils kurzer Beschreibung (Stichpunkte oder 1 bis 2 Sätze pro Skizze)

8 Qualitätsziele

Der Code soll gut kommentiert und klar strukturiert sein, sodass sich neue Entwicklerinnen und Entwickler gut in dem Projekt zurechtfinden und es leicht erweitern und ändern können. Um dies zu erleichtern, wird außerdem ein Developerhandbuch erstellt.

Das Ergebnis des Projekts soll eine hohe Benutzerfreundlichkeit haben und für Anwenderinnen und Anwender wird zusätzlich ein Benutzerhandbuch bereitgestellt.